# A Tutorial Introduction to Larkita

Karl Stroetmann

October 30, 2026

**Abstract**

Larkita is a parser generator for *Python*. It reads a context-free grammar and produces a canonical LR(1) parser that returns the abstract syntax tree of its input as a nested tuple. The shape of these tuples follows from the grammar, so there is no tree-building code to write. Furthermore, Larkita describes the LR(1) automaton it has generated in an output file, which makes conflicts easy to understand. If the grammar is LALR(1), Larkita can also produce the much smaller LALR(1) parser: On request, it merges states that only differ in their look-ahead tokens, as long as this is safe. This tutorial explains how to write grammars for Larkita, how the abstract syntax trees are built, how to process them with structural pattern matching, and how to deal with syntax errors and conflicts.

## Contents

**1 Introduction** **2**

1.1 Installation . . . 3

1.2 The examples . . . 3

**2 A First Parser** **3**

**3 Writing Grammars** **5**

3.1 Rules . . . 5

3.2 Terminals . . . 5

3.3 The scanner . . . 6

**4 How the Abstract Syntax Tree is Built** **7**

4.1 The head . . . 7

4.2 Pass-through . . . 8

4.3 The rule name as head . . . 8

4.4 Repetitions and groups . . . 8

4.5 Optional parts . . . 9

4.6 Ignoring literals . . . 9

4.7 Aliases . . . 10

4.8 Summary . . . . . 10

**5 Processing the Abstract Syntax Tree 11**

**6 A Small Interpreter 14**

**7 Syntax Errors 20**

**8 The Output File 22**

**9 Conflicts 24**

9.1 Shift/reduce conflicts . . . . . 24

9.2 Reduce/reduce conflicts . . . . . 26

9.3 Priorities . . . . . 27

**10 Merging States 28**

**11 Reference 32**

11.1 The class `Larkita` . . . . . 32

11.2 Grammar commands . . . . . 32

11.3 Exceptions and warnings . . . . . 33

11.4 Differences from LARK . . . . . 33

# 1 Introduction

LARKITA is a light-weight version of the parser generator LARK. It differs from LARK in three ways:

1. By default, it generates a canonical LR(1) parser. These parsers can handle more grammars than LALR(1) parsers, and when a grammar has a conflict, the LR(1) states show directly where the conflict comes from. With the option «`merge_states=True`», LARKITA merges states that only differ in their look-ahead tokens, as long as this causes no new conflicts and does not change how the input is split into tokens. If the grammar is LALR(1) and the basic scanner is used, the result is the LALR(1) parser, which has far fewer states than an LR(1) parser (see Section 10).

2. The parser returns the abstract syntax tree as a nested tuple. For example, with the grammar shown in Figure 1, the string «`x := a + 3;`» is turned into the tuple

   ```
   (':=', ('IDENTIFIER', 'x'),
          ('+', ('IDENTIFIER', 'a'), ('NUMBER', '3'))).
   ```

   Tuples like these are easy to print and easy to process with the «`match`» statement of *Python*.

3. Every time a parser is generated, LARKITA writes an output file that lists all LR(1) states. Every item of a state is annotated with its look-ahead tokens.

Grammars are written in the EBNF notation of Lark, with a few additions that control the shape of the abstract syntax trees.

### 1.1 Installation

Larkita is written in pure *Python* and needs no other packages. It can be installed from PyPI:

```
pip install larkita
```

Larkita runs with *Python* 3.12 or newer. Its source code is on GitHub.

### 1.2 The examples

All grammars and programs shown in this tutorial are contained in the directory `tutorial/examples` of the Larkita repository. The listings in this document are read from these files and the outputs shown are the outputs that the programs actually produce: The script `run_examples.py` runs all examples and stores their outputs in the directory `examples/output`, from where they are included into this document.

## 2 A First Parser

The file `expr.g` shown in Figure 1 contains a grammar for assignments and arithmetic expressions.

```
1  // A grammar for assignments and arithmetic expressions.
2
3  stmnt: IDENTIFIER ":=" expr ";"    %ignore ";"
4       | expr ";"
5
6  expr: expr "+" product
7      | expr "-" product
8      | product
9
10 product: product "*" factor
11        | product "/" factor
12        | factor
13
14 factor: "(" expr ")"    %ignore "(", ")"
15       | NUMBER
16       | IDENTIFIER
17
18 %import common.CNAME -> IDENTIFIER
19 %import common.NUMBER
20 %import common.WS
21 %ignore WS
```

Figure 1: The grammar `expr.g` for assignments and arithmetic expressions.

- Line 1 is a comment. Comments start with «//» and extend to the end of the line.
- A rule consists of the name of a syntactical variable, a colon, and a list of alternatives that are separated by the character « | ». The names of syntactical variables are written in lower case. The first rule defines the start symbol of the grammar, which is `stmnt` in this case.
- Strings enclosed in double quotes like `":="` or `"+"` are literals: They stand for exactly the characters written between the quotes.
- Names written in upper case like `IDENTIFIER` and `NUMBER` denote terminals. These terminals are defined in line 18 – 20: The command

  ```
  %import common.CNAME -> IDENTIFIER
  ```

  imports the terminal `CNAME` from the library `common.lark`, which is part of Larkita, and renames it into `IDENTIFIER`. `CNAME` matches names as they are used in the programming language C. Similarly, `NUMBER` matches integers and floating point numbers, and `WS` matches white space.
- Line 21 tells the scanner to skip white space.
- The annotations «`%ignore ";"`» in line 3 and «`%ignore "(", ")"`» in line 14 are specific to Larkita. They tell Larkita that the semicolon of an assignment and the parentheses should not appear in the abstract syntax tree. The semicolon in line 4 is kept: It is the only literal of this alternative, and without it the abstract syntax tree of the statement would just be the tree of the expression. We will discuss «`%ignore`» in Section 4.

The program `first.py` in Figure 2 generates a parser from this grammar and parses a string.

```
1 from pprint import pprint
2 from larkita import Larkita
3
4 parser = Larkita.open('expr.g')
5 ast = parser.parse('x := (a + 3) * b;')
6 pprint(ast, width=60)
```

Figure 2: The program `first.py` parses a string with the grammar `expr.g`.

- The class method «`Larkita.open`» reads the grammar from a file and returns a parser. If the grammar is given as a string, we use the constructor instead: «`Larkita(grammar)`».
- The method `parse` takes the string to parse and returns its abstract syntax tree. We print this tree with the function `pprint`, which inserts line breaks and indentation:

```
(':=',
 ('IDENTIFIER', 'x'),
 ('*',
  ('+', ('IDENTIFIER', 'a'), ('NUMBER', '3')),
  ('IDENTIFIER', 'b')))
```

Every node of this tree is a tuple. The first component of a tuple is called its head: It contains the literals of the rule that has been used, except for the ignored ones. The remaining components are

the children. For example, the tuple «`('+', ('IDENTIFIER', 'a'), ('NUMBER', '3'))`» represents the expression «`a + 3`». Its head is the operator «`'+'`» and its children represent the operands. A token like «`a`» is represented as a pair consisting of the name of the terminal and the text that has been read. Section 4 explains the construction of the abstract syntax tree in detail.

When Larkita generates a parser, it also writes an output file that describes the parser. As the grammar has been read from the file `expr.g`, this file is called `expr.out` and is written next to the grammar. If the grammar is given as a string, the output file is called `grammar.out` and is written into the current directory. The parameter `out_file` sets another path. Section 8 explains the output file.

# 3 Writing Grammars

Larkita reads grammars in the EBNF notation of Lark. This section summarizes this notation.

## 3.1 Rules

The right hand side of a rule can use the following operators:

| Notation | Meaning |
|---|---|
| «`a \| b`» | either a or b |
| «`(a b)`» | grouping |
| «`[a b]`» | optional: a b or nothing |
| «`a?`» | optional: a or nothing |
| «`a*`» | zero or more repetitions of a |
| «`a+`» | one or more repetitions of a |
| «`a ~ 3`» | exactly three repetitions of a |
| «`a ~ 2..4`» | between two and four repetitions of a |

Larkita translates these operators into ordinary grammar rules. The output file lists the resulting rules.

Alternatives are usually written on lines of their own. An alternative may be empty, so the rule

```
args: expr ("," expr)*
    |
```

describes a possibly empty list of expressions. A rule can be given a priority by appending a dot and a number to its name, as in «`expr.2: ...`». Priorities are used to resolve reduce/reduce conflicts, see Section 9.3.

## 3.2 Terminals

There are three ways to specify the tokens of a language:

1. A literal is a string enclosed in double quotes, e.g. `"while"`. If the letter `i` is appended, as in «`"while"i`», upper and lower case are not distinguished.
2. A regular expression is enclosed in slashes, e.g. «`/[0-9]+/`». It can be followed by flags, e.g. «`/begin/i`». The syntax is the syntax of the *Python* module `re`.

3. A terminal is defined by a rule whose name is written in upper case. Its right hand side may use literals, regular expressions, other terminals, and the operators shown above. For example:

```
DIGIT:  "0".."9"
INT:    DIGIT+
FLOAT:  INT "." INT
STRING: /"[^"]*"/
```

   The notation `"0".."9"` denotes a range of characters. A terminal can be given a priority, as in «`FLOAT.2: INT "." INT`». If two terminals match the input, the scanner prefers the terminal with the higher priority.

The library `common.lark` defines the following terminals, among others:

| Terminal | Matches |
|---|---|
| INT, SIGNED_INT | natural numbers and integers |
| FLOAT, NUMBER | floating point numbers; integers or floating point numbers |
| CNAME | names, i.e. a letter or an underscore followed by letters, digits, and underscores |
| ESCAPED_STRING | strings in double quotes |
| WS, WS_INLINE, NEWLINE | white space; white space without line breaks; line breaks |
| CPP_COMMENT, C_COMMENT | comments of the form «`// ...`» and «`/* ... */`» |
| SH_COMMENT | comments of the form «`# ...`» |

They are imported with «`%import common.NAME`». The command

```
%import common.NAME -> NEW_NAME
```

also renames the terminal.

## 3.3 The scanner

The scanner splits the input into tokens. The command «`%ignore NAME`» tells the scanner to skip the tokens of the terminal NAME, e.g. white space or comments. The method `lex` runs only the scanner, which is useful when a grammar does not work as expected. Figure 3 shows an example.

```
1 from larkita import Larkita
2
3 parser = Larkita.open('expr.g')
4 for t in parser.lex('x := 3 * (y + 1);'):
5     print(f'{t.type:10} {t.value!r:5} {t.line}:{t.column}')
```

Figure 3: The program `lex.py` splits a string into tokens.

The method `lex` returns an iterator over the tokens. Every token has a type, a value, a line, and a column. The program prints them in the form `line:column`:

```
IDENTIFIER 'x'    1:1
__ANON_0   ':='   1:3
NUMBER     '3'    1:6
STAR       '*'    1:8
LPAR       '('    1:10
IDENTIFIER 'y'    1:11
PLUS       '+'    1:13
NUMBER     '1'    1:15
RPAR       ')'    1:16
SEMICOLON  ';'    1:17
```

Larkita invents names for the terminals that are given as literals, e.g. STAR for `"*"`. If a literal has no obvious name, it is called «__ANON_0», «__ANON_1», etc. The output file and the error messages show literals in quotes instead.

Keywords are handled automatically: If a literal like `"while"` can also be matched by the terminal of the identifiers, the string «`while`» is recognized as the keyword, but «`whiles`» or «`while1`» are still recognized as identifiers.

By default, the scanner is contextual: In every state of the parser, it only tries to match those terminals that the parser can accept in this state. This avoids many collisions between terminals. With the option «`lexer='basic'`», the scanner always tries all terminals.

## 4 How the Abstract Syntax Tree is Built

This section explains how Larkita builds the abstract syntax tree. Every example shows a small grammar, followed by some strings and their abstract syntax trees. The examples are produced by the program `shapes.py`, which adds the following terminal definitions to every grammar:

```
%import common.CNAME -> NAME
%import common.INT -> NUM
%import common.WS
%ignore WS
```

### 4.1 The head

When the parser reduces with an alternative, the string literals of this alternative form the head of a new tuple. All other symbols of the alternative provide the children of the tuple: A syntactical variable contributes its abstract syntax tree, while a named terminal contributes a pair of the form «`(NAME, text)`», where `text` is the text that has been read. This text is never converted, so the number 1 is represented as «`('NUM', '1')`». If an alternative contains exactly one literal, the head is this literal:

```
sum: NUM "+" NUM

'1 + 2' ⇒ ('+', ('NUM', '1'), ('NUM', '2'))
```

If an alternative contains several literals, the head is the tuple of these literals in the order in which

they occur in the alternative:

```
assign: NAME ":=" NUM ";"

'x := 1;' ⇒ ((':=', ';'), ('NAME', 'x'), ('NUM', '1'))
```

## 4.2 Pass-through

If an alternative has no head and consists of a single symbol, then no new tuple is built. Instead, the abstract syntax tree of this symbol is returned. This rule is called pass-through. It guarantees that alternatives like «`sum: atom`» leave no trace in the abstract syntax tree:

```
sum: sum "+" atom
   | atom
atom: NUM
    | NAME

'42'
    ⇒ ('NUM', '42')
'x + 1'
    ⇒ ('+', ('NAME', 'x'), ('NUM', '1'))
'1 + 2 + 3'
    ⇒ ('+', ('+', ('NUM', '1'), ('NUM', '2')), ('NUM', '3'))
```

The string «42» is parsed with the alternatives «`sum: atom`» and «`atom: NUM`», but the result is just the pair «`('NUM', '42')`».

## 4.3 The rule name as head

If an alternative has no head and is not a pass-through, the name of the rule is used as head:

```
program: stmnt*
stmnt: NAME ";"

'a; b;' ⇒ ('program', (';', ('NAME', 'a')), (';', ('NAME', 'b')))
'a;'    ⇒ ('program', (';', ('NAME', 'a')))
''      ⇒ ('program',)
```

Pass-through is decided by the grammar, not by the input: The alternative «`program: stmnt*`» always builds a tuple, even if the input contains only one statement. For the empty string, it builds the tuple «`('program',)`», which has no children.

## 4.4 Repetitions and groups

The children produced by repetitions and groups are inserted into the tuple of the alternative in which they occur. Literals that occur inside repetitions, optional parts, and groups belong to the head:

```
list: "[" NUM ("," NUM)* "]"

'[1, 2, 3]'
    ⇒ (('[', ',', ']'), ('NUM', '1'), ('NUM', '2'), ('NUM', '3'))
'[1]'
    ⇒ (('[', ',', ']'), ('NUM', '1'))
```

Note that the head is determined by the grammar, not by the input: The head of the abstract syntax tree of «`[1]`» contains a comma, although no comma has been read. Therefore, all lists produced by this alternative have the same head.

## 4.5 Optional parts

If an optional part is missing, it is simply left out:

```
ret: "return" [NUM] ";"

'return 1;' ⇒ (('return', ';'), ('NUM', '1'))
'return;'   ⇒ (('return', ';'),)
```

There is no placeholder like `None`, so the number of children tells whether the optional part has been present.

## 4.6 Ignoring literals

Some literals are needed for parsing, but they carry no information once the abstract syntax tree has been built. For example, parentheses determine the structure of an expression, but this structure is already represented by the tuples. Writing «`%ignore`» followed by a list of literals at the end of an alternative removes these literals from the head of this alternative. In the following example, the alternative «`atom: "(" sum ")"`» therefore has no head and is a pass-through:

```
sum: sum "+" atom
   | atom
atom: "(" sum ")"    %ignore "(", ")"
    | NUM

'(1 + 2) + 3'
    ⇒ ('+', ('+', ('NUM', '1'), ('NUM', '2')), ('NUM', '3'))
'((1))'
    ⇒ ('NUM', '1')
```

If some literals should be removed from all alternatives of the grammar, the command «`%ignore:`» (with a colon) is used. In the following example, the alternative has no head left. As it is not a pass-through, the name of the rule becomes the head:

```
list: "[" NUM ("," NUM)* "]"
%ignore: "[", "]", ","

'[1, 2, 3]' ⇒ ('list', ('NUM', '1'), ('NUM', '2'), ('NUM', '3'))
'[1]'       ⇒ ('list', ('NUM', '1'))
```

Note that there are three different forms of «`%ignore`»:

| Form | Effect |
|---|---|
| «`%ignore WS`» | the scanner skips the tokens of the terminal `WS` |
| «`a: ... %ignore "(", ")"`» | the literals are removed from the head of this alternative |
| «`%ignore: "(", ")"`» | the literals are removed from the heads of all alternatives |

## 4.7 Aliases

An alias replaces the head of an alternative. It is written at the end of the alternative, either as a name or as a string: «`-> call`» and «`-> "call"`» are equivalent.

```
expr: NAME "(" NAME ")"     -> call
    | "-" NUM               -> "neg"
    | NUM "-" NUM

'f(x)'  ⇒ ('call', ('NAME', 'f'), ('NAME', 'x'))
'-1'    ⇒ ('neg', ('NUM', '1'))
'2 - 1' ⇒ ('-', ('NUM', '2'), ('NUM', '1'))
```

Aliases are useful when the same literals are used for different purposes. Here, the unary minus and the binary minus both use the literal `"-"`. Without the alias, both would produce the head «`'-'`». Although the number of children would still tell them apart, the alias `neg` makes the abstract syntax tree easier to read.

## 4.8 Summary

The following list summarizes the rules. Every alternative is followed by the abstract syntax tree that it produces, where `ast(x)` denotes the abstract syntax tree of `x`:

```
expr: expr "+" product
    ⇒ ('+', ast(expr), ast(product))
stmnt: IDENTIFIER ":=" expr ";"
    ⇒ ((':=', ';'), ('IDENTIFIER', name), ast(expr))
atom: IDENTIFIER
    ⇒ ('IDENTIFIER', name)
expr: product
    ⇒ ast(product)
factor: "(" expr ")"    %ignore "(", ")"
    ⇒ ast(expr)
atom: NAME "(" args ")"    -> "call"
    ⇒ ('call', ('NAME', name), ast(args))
args: expr ("," expr)*
    ⇒ (',', ast(expr_1), ..., ast(expr_n))
program: stmnt*
    ⇒ ('program', ast(stmnt_1), ..., ast(stmnt_n))
ret: "return" [expr] ";"
    ⇒ (('return', ';'),)  or  (('return', ';'), ast(expr))
```

## 5 Processing the Abstract Syntax Tree

Abstract syntax trees that are represented as nested tuples are best processed with the «`match`» statement of *Python*. As an example, we implement a simple calculator. Its grammar `calc.g`, which is shown in Figure 4, extends the grammar `expr.g`: A program is a sequence of statements, the statement «`print expr;`» prints the value of an expression, and there is a unary minus.

```
// A grammar for a simple calculator.

program: stmnt*

stmnt: IDENTIFIER ":=" expr ";"    %ignore ";"
     | "print" expr ";"            %ignore ";"

expr: expr "+" product
    | expr "-" product
    | product

product: product "*" factor
       | product "/" factor
       | factor

factor: "(" expr ")"    %ignore "(", ")"
      | "-" factor      -> "neg"
      | NUMBER
      | IDENTIFIER

%import common.CNAME -> IDENTIFIER
%import common.NUMBER
%import common.WS
%ignore WS
```

Figure 4: The grammar `calc.g` of a simple calculator.

The program `calc.py` in Figure 5 parses a small program, prints its abstract syntax tree, and executes it.

```python
from pprint import pprint
from larkita import Larkita

parser = Larkita.open('calc.g')

def evaluate(expr, values):
    match expr:
        case ('+', lhs, rhs):
            return evaluate(lhs, values) + evaluate(rhs, values)
        case ('-', lhs, rhs):
            return evaluate(lhs, values) - evaluate(rhs, values)
        case ('*', lhs, rhs):
            return evaluate(lhs, values) * evaluate(rhs, values)
        case ('/', lhs, rhs):
            return evaluate(lhs, values) / evaluate(rhs, values)
        case ('neg', arg):
            return -evaluate(arg, values)
        case ('NUMBER', digits):
            return float(digits)
        case ('IDENTIFIER', name):
            return values[name]

def execute(program):
    values = {}
    for stmnt in program[1:]:
        match stmnt:
            case (':=', ('IDENTIFIER', name), expr):
                values[name] = evaluate(expr, values)
            case ('print', expr):
                print(evaluate(expr, values))

text = '''
x := 2 * (3 + 4);
print x / -4;
'''
ast = parser.parse(text)
pprint(ast, width=60)
execute(ast)
```

Figure 5: The program `calc.py` implements the calculator.

- The function `evaluate` computes the value of an expression. Every «`case`» corresponds to an alternative of the grammar: The pattern «`('+', lhs, rhs)`» matches the tuples produced by the alternative «`expr: expr "+" product`» and binds the variables `lhs` and `rhs` to the abstract syntax trees of the operands. The alias `neg` distinguishes the unary minus from the binary minus.
- The function `execute` executes a program. As the head of the abstract syntax tree of a program is the rule name `program`, the statements are the components «`program[1:]`». As the

semicolons are ignored, the head of an assignment is «':='» and the head of a «print» statement is «'print'».

The program prints the following:

```
('program',
 (':=',
  ('IDENTIFIER', 'x'),
  ('*',
   ('NUMBER', '2'),
   ('+', ('NUMBER', '3'), ('NUMBER', '4')))),
 ('print',
  ('/', ('IDENTIFIER', 'x'), ('neg', ('NUMBER', '4')))))
-3.5
```

## 6 A Small Interpreter

Our next example is an interpreter for a small imperative language with assignments, «print» statements, «if» statements, «while» loops, and blocks. Its grammar `while.g` in Figure 6 shows how «%ignore:» and aliases keep the abstract syntax tree small.

```
1  // A grammar for a small imperative language.
2
3  program: stmnt*
4
5  stmnt: IDENTIFIER ":=" expr ";"
6       | "print" "(" expr ")" ";"
7       | "if" "(" cond ")" stmnt "else" stmnt    -> "if"
8       | "while" "(" cond ")" stmnt
9       | "{" stmnt* "}"                         -> block
10
11 cond: expr "==" expr
12     | expr "!=" expr
13     | expr "<"  expr
14     | expr "<=" expr
15
16 expr: expr "+" product
17     | expr "-" product
18     | product
19
20 product: product "*" factor
21        | product "/" factor
22        | product "%" factor
23        | factor
24
25 factor: "(" expr ")"
26       | NUMBER
27       | IDENTIFIER
28
29 %ignore: "(", ")", "{", "}", ";"
30
31 %import common.CNAME -> IDENTIFIER
32 %import common.INT -> NUMBER
33 %import common.CPP_COMMENT
34 %import common.WS
35 %ignore CPP_COMMENT
36 %ignore WS
```

Figure 6: The grammar `while.g` of a small imperative language.

Instead of ignoring literals in every alternative separately, line 29 removes parentheses, braces, and semicolons from all heads. Hence:

- An assignment «`x := e;`» is represented as

  `(':=', ('IDENTIFIER', 'x'), ast(e))`.

- A statement «`print(e);`» is represented as «`('print', ast(e))`».
- The head of an «`if`» statement would be «`('if', 'else')`». The alias in line 7 replaces it with «`'if'`», so an «`if`» statement is represented as «`('if', ast(c), ast(s1), ast(s2))`».

- A loop «`while (c) s`» is represented as «`('while', ast(c), ast(s))`».
- A block has no head left. Without the alias in line 9, the rule name `stmnt` would be used as head. With the alias, a block is represented as «`('block', ast(s1), ..., ast(sn))`».
- The alternative «`factor: "(" expr ")"`» in line 25 has no head and is therefore a pass-through.

Line 33 and 35 tell the scanner to skip comments. Figure 7 shows the interpreter.

```python
import operator
from pprint import pprint
from larkita import Larkita

parser = Larkita.open('while.g')

OPERATORS = {
    '+':  operator.add, '-':  operator.sub, '*': operator.mul,
    '/':  operator.floordiv, '%': operator.mod,
    '==': operator.eq, '!=': operator.ne,
    '<':  operator.lt, '<=': operator.le,
}

def evaluate(expr, values):
    match expr:
        case ('NUMBER', digits):
            return int(digits)
        case ('IDENTIFIER', name):
            return values[name]
        case (op, lhs, rhs):
            x, y = evaluate(lhs, values), evaluate(rhs, values)
            return OPERATORS[op](x, y)

def execute(stmnt, values):
    match stmnt:
        case ('program' | 'block', *body):
            for s in body:
                execute(s, values)
        case (':=', ('IDENTIFIER', name), expr):
            values[name] = evaluate(expr, values)
        case ('print', expr):
            print(evaluate(expr, values))
        case ('if', cond, then_part, else_part):
            if evaluate(cond, values):
                execute(then_part, values)
            else:
                execute(else_part, values)
        case ('while', cond, body):
            while evaluate(cond, values):
                execute(body, values)
```

Figure 7: The interpreter `while.py`, part 1: the evaluation of expressions and the execution of statements.

- As all binary operators produce tuples of the form «`(op, lhs, rhs)`», the function `evaluate` needs only one case for them. The dictionary OPERATORS maps every operator to the corresponding function of the module `operator`.
- In the function `execute`, the pattern «`('program' | 'block', *body)`» matches both

programs and blocks. The star collects the statements of the program or the block in the list body.

Figure 8 shows the first program that is executed by the interpreter. It computes the greatest common divisor of 1071 and 462 with Euclid's algorithm. Figure 9 shows what is printed: the abstract syntax tree of the program, followed by its output.

```
42 euclid = '''
43 // Euclid's algorithm
44 a := 1071;
45 b := 462;
46 while (b != 0) {
47     r := a % b;
48     a := b;
49     b := r;
50 }
51 print(a);
52 '''
53 ast = parser.parse(euclid)
54 pprint(ast, width=60)
55 execute(ast, {})
```

Figure 8: The interpreter `while.py`, part 2: Euclid's algorithm is parsed and executed.

```
('program',
 (':=', ('IDENTIFIER', 'a'), ('NUMBER', '1071')),
 (':=', ('IDENTIFIER', 'b'), ('NUMBER', '462')),
 ('while',
  ('!=', ('IDENTIFIER', 'b'), ('NUMBER', '0')),
  ('block',
   (':=',
    ('IDENTIFIER', 'r'),
    ('%', ('IDENTIFIER', 'a'), ('IDENTIFIER', 'b'))),
   (':=', ('IDENTIFIER', 'a'), ('IDENTIFIER', 'b')),
   (':=', ('IDENTIFIER', 'b'), ('IDENTIFIER', 'r')))),
 ('print', ('IDENTIFIER', 'a')))
21
```

Figure 9: The abstract syntax tree and the output of Euclid's algorithm.

The second program, which is shown in Figure 10, computes the number of steps that the Collatz sequence starting at 27 needs to reach 1. Its abstract syntax tree and its output are shown in Figure 11.

```
collatz = '''
// the number of steps of the Collatz sequence starting at 27
n := 27;
steps := 0;
while (n != 1) {
    if (n % 2 == 0) { n := n / 2; } else { n := 3 * n + 1; }
    steps := steps + 1;
}
print(steps);
'''
ast = parser.parse(collatz)
pprint(ast, width=60)
execute(ast, {})
```

Figure 10: The interpreter `while.py`, part 3: the Collatz sequence is computed.

```
('program',
 (':=', ('IDENTIFIER', 'n'), ('NUMBER', '27')),
 (':=', ('IDENTIFIER', 'steps'), ('NUMBER', '0')),
 ('while',
  ('!=', ('IDENTIFIER', 'n'), ('NUMBER', '1')),
  ('block',
   ('if',
    ('==',
     ('%', ('IDENTIFIER', 'n'), ('NUMBER', '2')),
     ('NUMBER', '0')),
    ('block',
     (':=',
      ('IDENTIFIER', 'n'),
      ('/', ('IDENTIFIER', 'n'), ('NUMBER', '2')))),
    ('block',
     (':=',
      ('IDENTIFIER', 'n'),
      ('+',
       ('*', ('NUMBER', '3'), ('IDENTIFIER', 'n')),
       ('NUMBER', '1'))))),
   (':=',
    ('IDENTIFIER', 'steps'),
    ('+', ('IDENTIFIER', 'steps'), ('NUMBER', '1'))))),
 ('print', ('IDENTIFIER', 'steps')))
111
```

Figure 11: The abstract syntax tree and the output of the Collatz program.

# 7 Syntax Errors

If the input contains a syntax error, the method `parse` raises an exception. All exceptions of LARKITA are subclasses of `LarkitaError`. There are two kinds of syntax errors:

- `UnexpectedCharacters` is raised by the scanner if none of the terminals matches the input.
- `UnexpectedToken` is raised by the parser if it reads a token that it cannot accept in its current state. The attribute `token` contains this token and the attribute `expected` contains the set of terminals that the parser would have accepted. If the input ends prematurely, the token has the type «$END».

Both exceptions are subclasses of `UnexpectedInput`. They provide the attributes `line` and `column`, which give the position of the error, and the method «`get_context(text)`», which returns the line containing the error together with a marker that points to the error. The program `errors.py` in Figure 12 checks three erroneous programs for the grammar `while.g`. Every program starts on the line after the opening quotes. The function `check` removes the newline in front of it, so the line numbers refer to the lines of the program. The program `errors.py` prints the following:

```
from larkita import Larkita, UnexpectedInput

parser = Larkita.open('while.g')

def check(text):
    text = text.removeprefix('\n')
    try:
        parser.parse(text)
        print('no syntax errors')
    except UnexpectedInput as e:
        name = type(e).__name__
        print(f'{name} at line {e.line}, column {e.column}:')
        print(e.get_context(text))

check('''
x := 1;
while (x < 10) {
    x := x + ;
}
''')

check('''
x := 1;
print(x # 2);
''')

check('''
x := 1;
print(x)
''')
```

Figure 12: The program `errors.py` reports the positions of syntax errors.

```
UnexpectedToken at line 3, column 14:
    x := x + ;
             ^

UnexpectedCharacters at line 2, column 9:
print(x # 2);
        ^

UnexpectedToken at line 2, column 8:
print(x)
       ^
```

In the last example, the semicolon after «`print(x)`» is missing. As the input ends prematurely, the error is reported at the position of the last token. The message of an exception describes the error in more detail. The program `message.py` in Figure 13 prints such a message:

```python
from larkita import Larkita, UnexpectedInput

parser = Larkita.open('while.g')
text = '''
x := 1;
while (x < 10) {
    x := x + ;
}
'''
try:
    parser.parse(text.removeprefix('\n'))
except UnexpectedInput as e:
    print(e)
```

Figure 13: The program `message.py` prints the message of a syntax error.

```
Unexpected token Token('SEMICOLON', ';') at line 3, column 14.
Expected one of:
    * IDENTIFIER
    * "("
    * NUMBER
Previous tokens: [Token('PLUS', '+')]
```

## 8 The Output File

When Larkita generates a parser, it writes a description of this parser into an output file. For the grammar `expr.g`, the output file `expr.out` begins as follows:

```
Canonical LR(1) automaton generated by Larkita

Grammar:      expr.g
Start symbol: stmnt
States:       38
Conflicts:    0 (0 unresolved)
```

Next, the file lists the rules of the grammar. Every rule has a number, and every rule is followed by the abstract syntax tree that a reduction with this rule produces:

```
Every rule is followed by the abstract syntax tree that a reduction produces.
"*name" stands for the list of children collected by the auxiliary rule
↪  "name".
"text" stands for the text of a named token.

  (0)   $root_stmnt → stmnt
  (1)   stmnt → IDENTIFIER ":=" expr ";"  ⇒ (":=", ("IDENTIFIER", text),
  ↪  ast(expr))
  (2)   stmnt → expr ";"                  ⇒ (";", ast(expr))
  (3)   expr → expr "+" product           ⇒ ("+", ast(expr), ast(product))
  (4)   expr → expr "-" product           ⇒ ("-", ast(expr), ast(product))
  (5)   expr → product                    ⇒ ast(product)
  (6)   product → product "*" factor      ⇒ ("*", ast(product), ast(factor))
  (7)   product → product "/" factor      ⇒ ("/", ast(product), ast(factor))
  (8)   product → factor                  ⇒ ast(factor)
  (9)   factor → "(" expr ")"             ⇒ ast(expr)
  (10)  factor → NUMBER                   ⇒ ("NUMBER", text)
  (11)  factor → IDENTIFIER               ⇒ ("IDENTIFIER", text)
```

Rule (0) is added by LARKITA: Its left hand side is the new start symbol «`$root_stmnt`». If the grammar uses operators like «`*`» or «`?`», the list contains the auxiliary rules that LARKITA has introduced for them. Then the file lists the conflicts of the grammar, if there are any. Finally, it describes all states of the parser. For example, state 2 is the state that the parser enters after it has read an identifier at the beginning of a statement:

```
State 2

    (1)   stmnt → IDENTIFIER • ":=" expr ";"  { $END }
    (11)  factor → IDENTIFIER •               { "*", "+", "-", "/", ";" }

    ":="  shift and go to state 8
    "*"   reduce using rule (11) factor → IDENTIFIER
    "+"   reduce using rule (11) factor → IDENTIFIER
    "-"   reduce using rule (11) factor → IDENTIFIER
    "/"   reduce using rule (11) factor → IDENTIFIER
    ";"   reduce using rule (11) factor → IDENTIFIER
```

- The first part of a state lists its items. An item is a rule with a dot «•» marking how much of the right hand side has already been read. It is followed by the set of look-ahead tokens: These are the tokens that can follow the left hand side of the rule when the parser reduces with this rule. Items that have been added by the closure operation are marked with a «+» at the beginning of the line. State 2 has no such items.
- The second part lists the actions of the state. For every token that the parser can accept in this state, it says whether the parser shifts the token or reduces with a rule. Furthermore, it lists the gotos, i.e. the states the parser enters after it has reduced to a syntactical variable.

State 2 shows how the parser uses the look-ahead token: After reading an identifier, the parser does not know yet whether the statement is an assignment or an expression. If the next token is `":="`, it shifts this token. If the next token is an operator or a semicolon, the identifier is part of an expression and the parser reduces with rule (11).

The parser itself also provides some information about the grammar: The attributes `rules`,

`terminals`, `conflicts`, and `parse_table` contain the rules, the terminals, the conflicts, and the parse table, while `out_file` is the path of the output file.

# 9 Conflicts

If the grammar is not LR(1), there are states in which the parser cannot decide which action to take: These states contain conflicts. A shift/reduce conflict occurs when the parser could either shift the next token or reduce with a rule. A reduce/reduce conflict occurs when it could reduce with two different rules.

## 9.1 Shift/reduce conflicts

The grammar `dangling.g` in Figure 14 describes nested «`if`» statements. It suffers from the dangling else problem: In a statement like «`if (a) if (b) x; else y;`» it is not clear whether the «`else`» belongs to the first or to the second «`if`».

```
// The dangling else problem.

stmnt: "if" "(" NAME ")" stmnt
     | "if" "(" NAME ")" stmnt "else" stmnt
     | NAME ";"

%ignore: "(", ")"

%import common.CNAME -> NAME
%import common.WS
%ignore WS
```

Figure 14: The grammar `dangling.g` has a shift/reduce conflict.

LARKITA resolves shift/reduce conflicts in favour of shifting. In addition, it issues a warning of the class `ConflictWarning`. The program `dangling.py` in Figure 15 catches this warning and prints it:

```
import warnings
from pprint import pprint
from larkita import Larkita

with warnings.catch_warnings(record=True) as caught:
    warnings.simplefilter('always')
    parser = Larkita.open('dangling.g')
for warning in caught:
    print(f'{warning.category.__name__}: {warning.message}')

pprint(parser.parse('if (a) if (b) x; else y;'), width=60)
```

Figure 15: The program `dangling.py` catches the warning about the conflict.

```
ConflictWarning: 1 shift/reduce conflict(s) were resolved as shift, see
 ↪  dangling.out
('if',
 ('NAME', 'a'),
 (('if', 'else'),
  ('NAME', 'b'),
  (';', ('NAME', 'x')),
  (';', ('NAME', 'y'))))
```

Shifting means that an «`else`» belongs to the innermost «`if`» that has no «`else`» yet, which is the rule used by most programming languages. The output file `dangling.out` lists the conflict:

```
State 17, shift/reduce conflict on "else", resolved as shift
    - shift and go to state 18
    - reduce using rule (1) stmnt → "if" "(" NAME ")" stmnt
```

The conflict is also shown in the state where it occurs:

```
State 17

    (1)  stmnt → "if" "(" NAME ")" stmnt •              { "else", $END }
    (2)  stmnt → "if" "(" NAME ")" stmnt • "else" stmnt  { "else", $END }

    "else"  shift and go to state 18
    $END    reduce using rule (1) stmnt → "if" "(" NAME ")" stmnt

    shift/reduce conflict on "else", resolved as shift
        - shift and go to state 18
        - reduce using rule (1) stmnt → "if" "(" NAME ")" stmnt
```

After reading «`if (a) if (b) x;`», the parser could either reduce the inner «`if`» statement with rule (1) or shift the token `"else"`. The look-ahead token `"else"` is contained in the look-ahead sets of both items.

The warnings are handled by the module `warnings` of *Python*. For example, the statement

```
warnings.simplefilter('error', ConflictWarning)
```

turns these warnings into exceptions, so that every shift/reduce conflict is reported as an error, while «`warnings.simplefilter('ignore', ConflictWarning)`» suppresses them.

## 9.2 Reduce/reduce conflicts

LARKITA treats reduce/reduce conflicts as errors: It raises a `GrammarError`. The grammar `look-ahead.g` in Figure 16 is unambiguous, but it is not LR(1).

```
// An unambiguous grammar that is not LR(1).

a: b "U" "V"
 | c "U" "W"
b: "X"
c: "X"
```

Figure 16: The grammar `look-ahead.g` is unambiguous, but it is not LR(1).

After reading `"X"`, the parser has to decide whether `"X"` is a `b` or a `c`. This depends on whether `"U"` is followed by `"V"` or by `"W"`, but the parser can only look at one token, namely `"U"`. The program `look-ahead.py` in Figure 17 tries to generate a parser for this grammar and prints the message of the exception. Afterwards, it generates a parser for the grammar `look-ahead-solved.g` discussed below and parses two strings:

```
from larkita import Larkita, GrammarError

try:
    parser = Larkita.open('look-ahead.g')
except GrammarError as e:
    print(e)

parser = Larkita.open('look-ahead-solved.g')
print(parser.parse('XUV'))
print(parser.parse('XUW'))
```

Figure 17: The program `look-ahead.py` reports the reduce/reduce conflict of `look-ahead.g`.

```
The grammar has 1 unresolved reduce/reduce conflict(s), see look-ahead.out:
  * state 4 on "U"
('V', (('X', 'U'),))
('W', (('X', 'U'),))
```

The output file is written even if the grammar has conflicts, so it can be used to find the cause of the problem:

```
State 4

    (3)  b → "X" •  { "U" }
    (4)  c → "X" •  { "U" }

    "U"  reduce using rule (3) b → "X"

    reduce/reduce conflict on "U", NOT RESOLVED (this is an error)
        - reduce using rule (3) b → "X"
        - reduce using rule (4) c → "X"
```

This conflict is solved by rewriting the grammar, so that the decision can be postponed until the token `"U"` has been read. The result is the grammar `look-ahead-solved.g` shown in Figure 18.

```
1  // The grammar of look-ahead.g, rewritten so that it is LR(1).
2
3  a: b "V"
4   | c "W"
5  b: "X" "U"
6  c: "X" "U"
```

Figure 18: The grammar `look-ahead-solved.g` avoids the conflict of `look-ahead.g`.

## 9.3 Priorities

A reduce/reduce conflict can also be resolved by giving one of the rules a higher priority. In the grammar `priority.g` in Figure 19, a statement consisting of a single name could be both a declaration of a type without a variable and an expression. The priority 2 in line 9 resolves this conflict in favour of the expression. The program `priority.py` in Figure 20 parses a declaration and an expression:

```
1  // A reduce/reduce conflict that is resolved by a priority.
2
3  stmnt: decl ";"
4       | expr ";"
5
6  decl: NAME NAME      -> "decl"
7      | NAME           -> "type"
8
9  expr.2: NAME         -> "var"
10
11 %import common.CNAME -> NAME
12 %import common.WS
13 %ignore WS
```

Figure 19: The grammar `priority.g` resolves a reduce/reduce conflict with a priority.

```python
from larkita import Larkita

parser = Larkita.open('priority.g')
print(parser.parse('int x;'))
print(parser.parse('x;'))
```

Figure 20: The program `priority.py` parses a declaration and an expression.

```
(';', ('decl', ('NAME', 'int'), ('NAME', 'x')))
(';', ('var', ('NAME', 'x')))
```

The output file lists the conflict together with its resolution:

```
State 4, reduce/reduce conflict on ";", resolved by priority: reduce using
↪  rule (5)
    - reduce using rule (4) decl → NAME
    - reduce using rule (5) expr → NAME
```

Priorities should be used with care: The rule with the lower priority is never used to reduce in the states where the conflict occurs. Usually, it is better to rewrite the grammar.

## 10 Merging States

Canonical LR(1) automata can have many states. For example, the parser for the grammar `while.g` of Section 6 has 142 states. Many of these states have the same core, i.e. they contain the same items and only differ in the look-ahead sets of these items. An LALR(1) parser merges all states with the same core, which yields far fewer states, but this can cause reduce/reduce conflicts that the canonical LR(1) parser does not have. With the option «`merge_states=True`», LARKITA merges states with the same core, as long as this is safe:

1. Merging must not cause a new conflict, neither in the merged states nor in their successors. As two merged states have one successor for every symbol, their successors have to be merged, too.
2. Merging must not change how the input is split into tokens. As explained in Section 3.3, the contextual scanner only tries the terminals that the parser can accept in its current state. If the look-ahead tokens of one state are added to another state, the scanner might read some text as a different token.

The program `merge.py` in Figure 21 generates three parsers for the grammar `while.g`: a canonical parser, a parser with merged states, and a parser with merged states that uses the basic scanner. It prints the number of states of every parser and parses the string «`x := 1; else := 2;`»:

```python
from larkita import Larkita, UnexpectedInput

canonical = Larkita.open('while.g')
merged = Larkita.open('while.g', merge_states=True,
                      out_file='while-merged.out')
basic = Larkita.open('while.g', merge_states=True, lexer='basic',
                     out_file='while-basic.out')

parsers = [('canonical', canonical),
           ('merged', merged),
           ('merged, basic lexer', basic)]
for name, parser in parsers:
    print(f'{name}: {len(parser.parse_table.states)} states')
    try:
        print('   ', parser.parse('x := 1; else := 2;')[2])
    except UnexpectedInput as e:
        print('   ', type(e).__name__, 'at column', e.column)
```

Figure 21: The program `merge.py` compares parsers with and without merged states.

```
canonical: 142 states
    (':=', ('IDENTIFIER', 'else'), ('NUMBER', '2'))
merged: 81 states
    (':=', ('IDENTIFIER', 'else'), ('NUMBER', '2'))
merged, basic lexer: 56 states
    UnexpectedToken at column 9
```

Merging reduces the number of states from 142 to 81. The output file `while-merged.out` describes the merged states. For example, state 3 has been obtained by merging the LR(1) states 3 and 16. Its look-ahead set is the union of their look-ahead sets:

```
State 3   (merged from the LR(1) states 3, 16)

    (3)  stmnt → IDENTIFIER • ":=" expr ";"  { "if", "print", "while", "{",
    ↪  "}", IDENTIFIER, $END }

    ":="  shift and go to state 10
```

A new section of the output file explains why some states with the same core have been kept apart. It begins as follows:

```
States with the same core have been merged, unless this would cause
a conflict: 142 LR(1) states have been merged into 81 states.

The following states have the same core, but they have been kept
apart.  Either merging them would cause a conflict, or one of them
accepts a token that collides with a token of the other one, i.e.
the scanner could read the same text as either of these tokens.

States 15 and 65
    on "else":
        state 15 does not accept "else" (collides with IDENTIFIER)
        state 65 reduces using rule (8) stmnt → "{" "}"
    on IDENTIFIER:
        state 15 reduces using rule (8) stmnt → "{" "}"
        state 65 does not accept IDENTIFIER (collides with "else")
```

After the statement «{ }», state 15 expects the next statement, which may start with an identifier, while state 65 expects the keyword «`else`», because the block is the then-branch of an «`if`» statement. In state 15, the contextual scanner reads the string «`else`» as an identifier, because the keyword `"else"` cannot occur here. If the two states were merged, the scanner would read «`else`» as a keyword in both contexts. Hence, these states are kept apart. The output file lists five more pairs of this kind, one for every other kind of statement. For example, the parser reaches state 27 after an assignment like «`x := 1;`» that is not followed by «`else`». Therefore, both the canonical parser and the parser with merged states read the string «`else`» in «`x := 1; else := 2;`» as an identifier. All other states that have been kept apart only differ in their successors.

The basic scanner always tries all terminals, so merging cannot change the tokens. Therefore, with the basic scanner all states with the same core of the grammar `while.g` are merged and the result is the LALR(1) automaton with 56 states. However, the basic scanner always reads «`else`» as a keyword, so this parser rejects «`else := 2;`». The basic scanner without merging would reject it, too. The output file `while-basic.out` reports:

```
States with the same core have been merged, unless this would cause
a conflict: 142 LR(1) states have been merged into 56 states.
All states with the same core have been merged, so this is the
LALR(1) automaton.
```

The grammar `not-lalr.g` in Figure 22 is LR(1), but it is not LALR(1).

```
1 // A grammar that is LR(1), but not LALR(1).
2
3 s: "v" a "y"
4  | "w" b "y"
5  | "v" b "z"
6  | "w" a "z"
7
8 a: "x"    -> a
9 b: "x"    -> b
```

Figure 22: The grammar `not-lalr.g` is LR(1), but it is not LALR(1).

After reading `"v" "x"` and `"w" "x"`, the parser reaches two states with the same core. In the first state, `"x"` has to be reduced to a if the next token is `"y"`, and to b if it is `"z"`. In the second state, it is the other way round. Merging these states would therefore cause two reduce/reduce conflicts, so they are kept apart. The program `not-lalr.py` in Figure 23 generates a parser with merged states and parses four strings:

```
from larkita import Larkita

parser = Larkita.open('not-lalr.g', merge_states=True)
for text in ['vxy', 'wxy', 'vxz', 'wxz']:
    print(text, parser.parse(text))
```

Figure 23: The program `not-lalr.py` generates a parser for `not-lalr.g` with merged states.

```
vxy (('v', 'y'), ('a',))
wxy (('w', 'y'), ('b',))
vxz (('v', 'z'), ('b',))
wxz (('w', 'z'), ('a',))
```

```
States with the same core have been merged, unless this would cause
a conflict: 14 LR(1) states have been merged into 14 states.

The following states have the same core, but they have been kept
apart.  Either merging them would cause a conflict, or one of them
accepts a token that collides with a token of the other one, i.e.
the scanner could read the same text as either of these tokens.

States 6 and 9
    on "y":
        state 6 reduces using rule (5) a → "x"
        state 9 reduces using rule (6) b → "x"
    on "z":
        state 6 reduces using rule (6) b → "x"
        state 9 reduces using rule (5) a → "x"
```

As these are the only states with the same core, merging does not reduce the number of states for this grammar.

A parser with merged states accepts the same strings and returns the same abstract syntax trees as the canonical parser, and it detects syntax errors at the same tokens. However, a merged state may reduce on a look-ahead token that is an error in some of the states it has been merged from. Then the parser performs some reductions before it detects the error, and the list of expected tokens in the error message may differ. As the output file of the canonical parser is easier to understand, states are only merged if this is requested.

# 11 Reference

## 11.1 The class `Larkita`

```
Larkita(grammar, *, start=None, lexer='contextual', import_paths=(),
        source_path=None, out_file=None, regex=False, g_regex_flags=0,
        merge_states=False)
```

- `grammar`: the grammar as a string or as an open file.
- `start`: the start symbol. By default, the first rule defines the start symbol.
- `lexer`: either «`'contextual'`» or «`'basic'`».
- `import_paths`: additional directories that are searched for imported grammars.
- `source_path`: the file name of the grammar. It is used for relative imports and to name the output file.
- `out_file`: the path of the output file.
- `regex`: if `True`, the module `regex` is used instead of the module `re`.
- «`g_regex_flags`»: flags that are applied to all regular expressions, e.g. «`re.IGNORECASE`».
- «`merge_states`»: if `True`, states with the same core are merged, as long as this is safe (see Section 10).

«`Larkita.open(filename, rel_to=None, **options)`» reads the grammar from a file. If «`rel_to`» is given, the file name is interpreted relative to the directory of the file «`rel_to`», so «`Larkita.open('expr.g', rel_to=__file__)`» finds a grammar that is stored next to the program. The other options are passed to the constructor.

A parser provides the following methods and attributes:

- «`parse(text)`» returns the abstract syntax tree of `text`.
- «`lex(text)`» returns an iterator over the tokens of `text`.
- `rules`, `terminals`, `conflicts`, `parse_table`, and «`out_file`» describe the generated parser.

## 11.2 Grammar commands

- «`%import common.NAME`» imports a terminal from the library `common.lark`.
- «`%import .file.NAME`» imports a rule or a terminal from the grammar `file.lark`, which is stored in the directory of the current grammar.
- «`-> NEW_NAME`» at the end of an import renames the imported symbol.
- «`%ignore NAME`» makes the scanner skip the terminal `NAME`.

- «`%ignore "a", "b"`» at the end of an alternative removes these literals from its head, while «`%ignore: "a", "b"`» removes them from all heads.
- «`%declare NAME`» declares a terminal that is not defined in the grammar.
- «`%override`» and «`%extend`» replace or extend a rule or a terminal that has been imported.

## 11.3 Exceptions and warnings

| Class | Meaning |
|---|---|
| `LarkitaError` | the base class of all exceptions of LARKITA |
| `GrammarError` | the grammar is erroneous or has an unresolved reduce/reduce conflict |
| `UnexpectedInput` | the base class of `UnexpectedCharacters` and `UnexpectedToken` |
| `UnexpectedCharacters` | no terminal matches the input |
| `UnexpectedToken` | the parser cannot accept the next token |
| `ConflictWarning` | a shift/reduce conflict has been resolved as shift |

## 11.4 Differences from LARK

Readers who know LARK should note the following differences:

- LARKITA always builds the canonical LR(1) automaton; an LALR(1) parser is obtained by merging states (see below). The Earley and CYK parsers of LARK are not available.
- The parser returns nested tuples. There are no trees, transformers, or visitors. Therefore, the rule modifiers «`?rule`» and «`!rule`» are not supported, and rules and terminals whose names start with an underscore are neither inlined nor filtered.
- Aliases and «`%ignore`» after an alternative and «`%ignore:`» control the heads of the tuples.
- Grammar templates, post-lexers like the `Indenter`, lexer callbacks, the interactive parser, and caching are not supported.
- Shift/reduce conflicts are resolved as shift with a warning, and every parser comes with an output file that describes the LR(1) automaton.
- By default, the states of the canonical LR(1) automaton are not merged. If the option «`merge_states=True`» is given, states with the same core are merged as far as this is safe.